51. Building Your Persistent Linux Lab

51. Building Your Persistent Linux Lab

Phase: Phase 0 — Orientation
Estimated time: 1–2 days
Slug: `51-lab-vm-practice`

Learning objectives

- Create a disposable VM snapshot workflow
- Document lab IP/users
- Separate 'break glass' practice VMs
- Automate lab provisioning lightly

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Linux skill comes from breaking things on purpose and fixing them. But the exercises that teach the most, such as editing fstab, changing firewall rules, resizing disks, and breaking SSH, are exactly the ones that can leave a machine unbootable or unreachable. If you only have one machine, you will avoid those exercises and plateau. A lab that you can reset in minutes removes the fear and makes you faster at real incidents.

The safety tool is the **snapshot**. Take one before every risky experiment, and revert when you finish or fail. Snapshots turn "I broke it" from a catastrophe into a two-minute reset.

Core concepts

Concept	Meaning
Hypervisor	software that runs VMs (UTM, VirtualBox, KVM, cloud)
Snapshot	frozen state of the VM disk (and optionally RAM)
Base image	clean installed VM you clone from
Host-only / NAT	virtual networks that keep the lab isolated
Console	out-of-band access when SSH or networking is broken
Break-glass VM	a VM you are explicitly allowed to destroy
Lab notes	a log of IPs, users, commands, and failures

```
your laptop (host)
 |
```

```
+-- hypervisor
      |-- vm-ubuntu (snapshot: clean, pre-firewall, pre-disk)
      |-- vm-alpine (contrast: musl, apk, OpenRC)
      |-- vm-break  (disposable: broken boot, bad fstab)
network: NAT for updates + host-only for lab-to-lab traffic
rule: never put prod credentials or real data in any lab VM
```

Worked examples

Example 1: a naming and notes convention.

```
lab-ubuntu   192.168.56.10  user: student   key: ~/.ssh/lab_ed25519
lab-alpine   192.168.56.11  user: student   key: ~/.ssh/lab_ed25519
lab-break    192.168.56.12  user: student   (disposable)
```

Keep this in a text file in your notes repository.

Example 2: snapshot workflow with VirtualBox command line.

```
VBoxManage snapshot lab-ubuntu take clean-install --description "fresh"
VBoxManage snapshot lab-ubuntu list
# ...break things...
VBoxManage controlvm lab-ubuntu poweroff
VBoxManage snapshot lab-ubuntu restore clean-install
```

UTM, VMware, Proxmox, and cloud providers have equivalent snapshot features in their UI or CLI.

Example 3: a separate key just for the lab.

```
ssh-keygen -t ed25519 -f ~/.ssh/lab_ed25519 -C "lab-only" -N ""
ssh-copy-id -i ~/.ssh/lab_ed25519.pub student@192.168.56.10
ssh -i ~/.ssh/lab_ed25519 student@192.168.56.10 hostname
```

Example 4: lightly automate setup so a rebuild is cheap.

```
#!/usr/bin/env bash
set -euo pipefail
sudo apt-get update
sudo apt-get install -y curl git jq dnsutils tcpdump nginx
sudo useradd -m -s /bin/bash practice || true
echo "lab provisioned $(date -Is)" | sudo tee /etc/motd
```

Example 5: a quick “what did I change” habit.

```
history | tail -50 > ~/notes/session-$(date +%F).txt
sudo cp -a /etc /root/etc-backup-$(date +%F)
```

Mini-lab

1. Create or pick a VM and install Ubuntu Server LTS.
2. Create a snapshot named `clean-install`.
3. Break something: add a bogus line to `/etc/fstab` and reboot, or flush firewall rules wrongly. Observe the symptom.

4. Use the hypervisor console, not SSH, to recover or to restore.
5. Restore the snapshot and confirm the problem is gone.
6. Add a second snapshot, `pre-firewall`, before the nftables module.
7. Write your lab inventory (names, IPs, users, snapshots) in a notes file.

Decision guide / failure modes

Situation	Choice
Risky disk, boot, or firewall exercise	snapshot first
Want to compare distro behavior	second VM, different family
Need to test destructive scripts	the break-glass VM only
Need real services reachable	host-only network plus forwarding
Forgot to snapshot	rebuild from base image, then automate

Failure modes: practicing firewall rules on your only machine without a console; having no snapshots; snapshots piling up until the disk is full; copying real production keys, tokens, or data into the lab; keeping a lab VM exposed to the internet with a weak password; and not writing down what you learned. Snapshots are not backups of valuable data, they are undo buttons.

Common mistakes

- Practicing firewall on the only machine you own without console
- No snapshots
- Mixing prod credentials into lab

Exercises

Work in `exercises/51-lab-vm-practice/`. Read that folder's `README.md` fully before coding.

```
cd exercises/51-lab-vm-practice
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `lab_safety_tool`. It prints the single lowercase word naming the feature the lesson says to use before every risky lab experiment so you can return to a known good state quickly. The test is a recall check and does not touch a hypervisor. After it passes, do the mini-lab for real: create a VM, take a snapshot, break `/etc/fstab` on purpose, recover through the console, and restore. Write down the commands or UI steps you used and the time the reset took. Note one rule you will follow about credentials in the lab.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/51-lab-vm-practice/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Why snapshots?
2. What two distros to compare?
3. Prod creds in lab risk?

Further reading

- https://ubuntu.com/download/server